

Unit-2

❖ Introduction about visual studio tools

- Visual Studio is Microsoft's integrated development environment (IDE) for building applications using various programming languages and platforms, including C# and the .NET ecosystem. It offers a range of editions catering to diverse developer needs, from individual hobbyists to large enterprises.
- **Visual Studio Community:**
- Cost: Free.
- Target Audience: Students, open-source developers, and individual developers, as well as small teams (under certain restrictions).
- Features: Provides a fully-featured IDE, essentially mirroring the functionality of the Professional edition, but with specific licensing constraints.
- Key takeaway: Great for learning, open-source contributions, and personal projects, offering a full range of development tools for C# and other .NET languages.
- ❖ **Visual Studio Professional:**
- Cost: Paid, subscription-based.
- Target Audience: Individual developers or small teams seeking professional development tools and services.
- Features: Offers all the features of the Community edition plus additional benefits, including more extensive testing tools, support for CodeLens in Visual Studio 2019 (a feature revealing code changes and other information directly in the editor).
- ❖ **Visual Studio versions have historically been linked to .NET Framework releases, each supporting specific C# versions with new language features and enhancements**
- ❖ **Early Versions (VS.NET 2002 - VS 2008):** Focused on establishing the foundations of C# and the .NET Framework, with features like generics (C# 2.0) and LINQ (C# 3.0) being introduced.
- ❖ **VS 2010 - VS 2013:** Continued adding features like asynchronous programming (C# 5.0) and supporting evolving .NET Framework versions.
- ❖ **VS 2015 - VS 2019:** Embraced the shift towards .NET Core and introduced features like string interpolation (C# 6.0), pattern matching (C# 7.0), and nullable reference types (C# 8.0).

- ❖ **Visual Studio 2022:** The latest major release, is a 64-bit application, offering significant performance improvements and enhanced support for .NET 6 (and later), C# 10.0 (and later), Hot Reload, and AI-powered development features with GitHub Copilot

Version	Year	Major C# Support	Key Features for C# Developers
VS 2005	2005	C# 2.0	Generics, anonymous methods, partial classes
VS 2008	2007	C# 3.0	LINQ, Lambda Expressions, Extension Methods
VS 2010	2010	C# 4.0	Dynamic typing, named and optional parameters
VS 2012	2012	C# 5.0	async and await, better async programming
VS 2013	2013	C# 5.0 (minor updates)	CodeLens, performance enhancements
VS 2015	2015	C# 6.0	Null-conditional operators, expression-bodied members
VS 2017	2017	C# 7.x	Tuples, pattern matching, local functions
VS 2019	2019	C# 8.0	Nullable reference types, async streams
VS 2022	2021	C# 10.0	Global usings, file-scoped namespaces
VS 2025 (Preview)	2025	C# 13.0 (expected)	Preview features, .NET 9 support, improved AI tools

❖ Create new window application with vb.net

- Create a New VB.NET Windows Forms Application
- Open Visual Studio 2019
- **Click:** Create a new project
- Search for Template: **"Windows Forms App (.NET Framework)"**
- **Language:** Visual Basic
- **Platform:** Windows
- **Project Type:** Desktop
- Click Next
- Configure Project
- Project Name: myvbapp
- **Location:** Choose where to save
- **Solution Name:** (Optional)
- **Framework Version:** e.g., **.NET Framework 4.7.2**
- Click Create

❖ Introduction about all categories of toolbox control

◆ 1. Common Controls

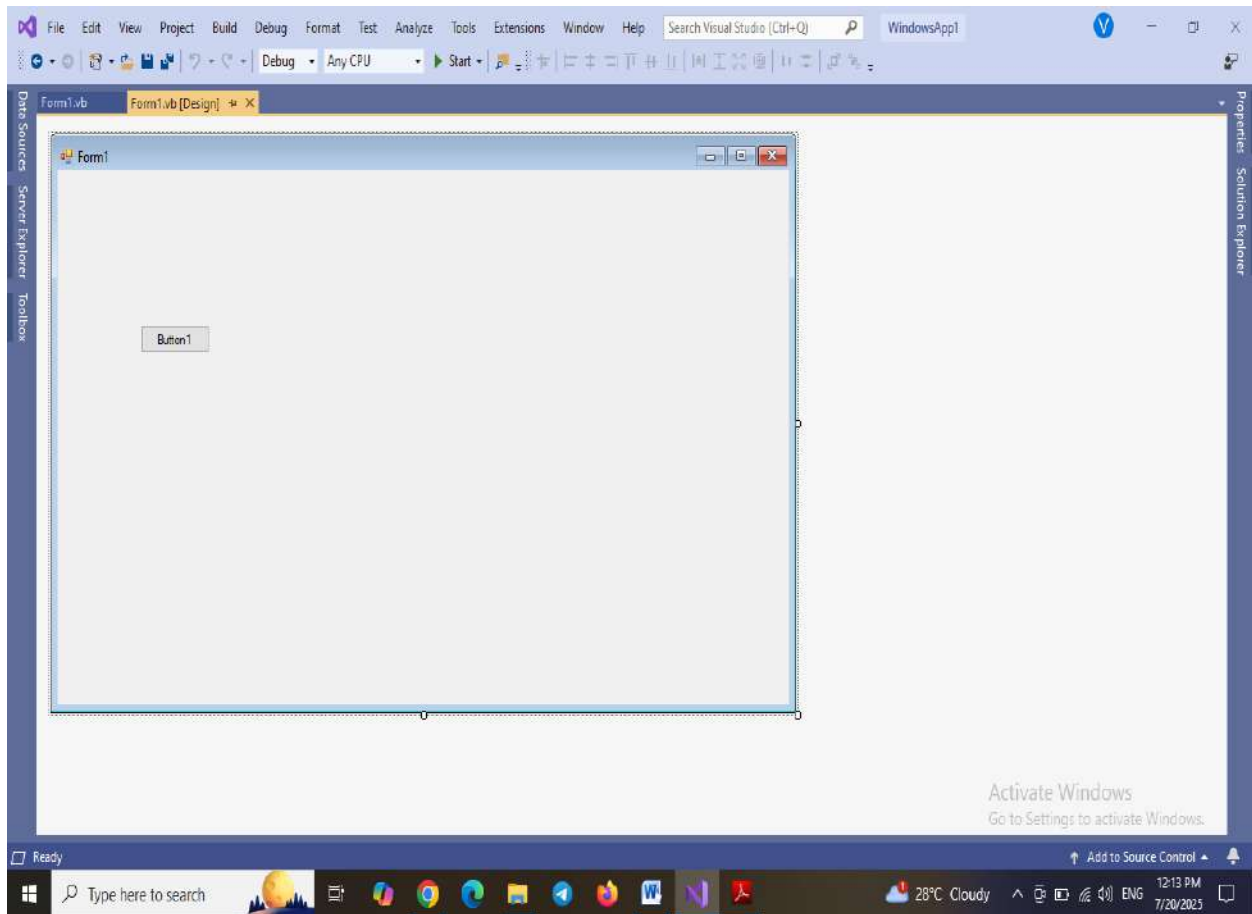
These are the most frequently used UI elements.

Control	Purpose
Button	Triggers an action when clicked
Label	Displays static text
TextBox	Takes user input
CheckBox	Allows users to select true/false options
RadioButton	Select one option from a group
ListBox	Displays a list of items
ComboBox	Drop-down list with optional text input
PictureBox	Displays images
LinkLabel	A clickable hyperlink-style label
ProgressBar	Shows task progress
DateTimePicker	Allows users to select dates and times

Used to group or organize other controls.

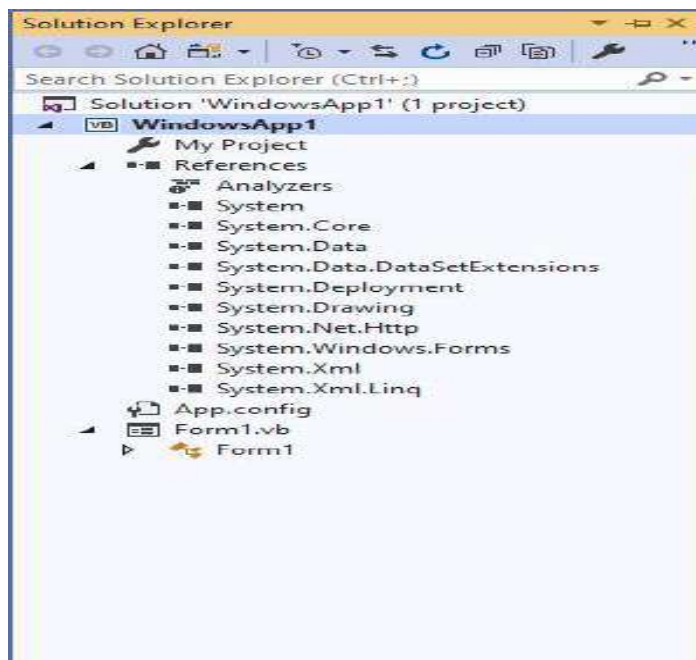
Control	Purpose
GroupBox	Groups related controls together
Panel	A simple container for controls
TabControl	Multi-tab interface
SplitContainer	Two resizable panels
FlowLayoutPanel	Auto-arranges controls in a flow layout
TableLayoutPanel	Arranges controls in a grid/table format

❖ Design Window:-



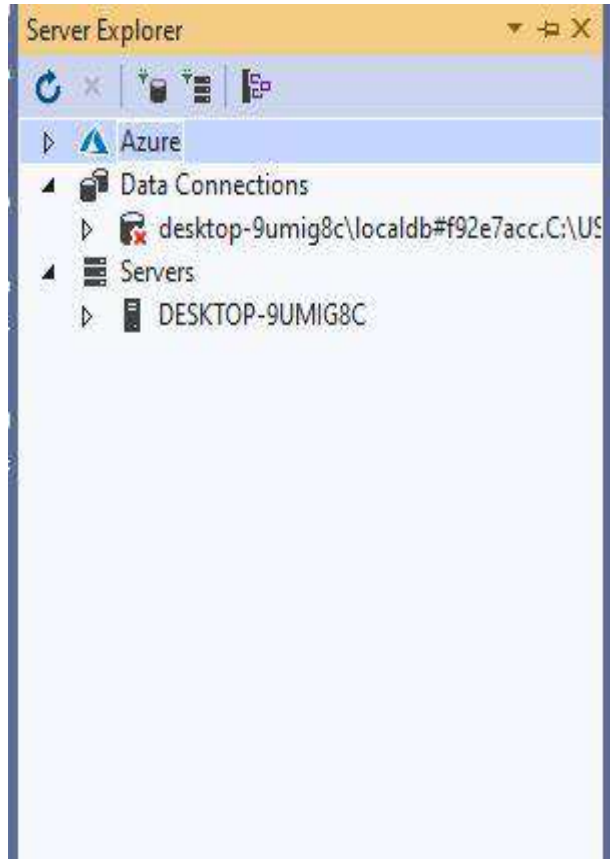
❖ Solution Explorer: -

- The **Solution Explorer** is a core tool in **Visual Studio** that lets you manage your **C# projects, files, references, and configurations**. It provides a **hierarchical view** of your **solution**, which can contain one or more **projects**.



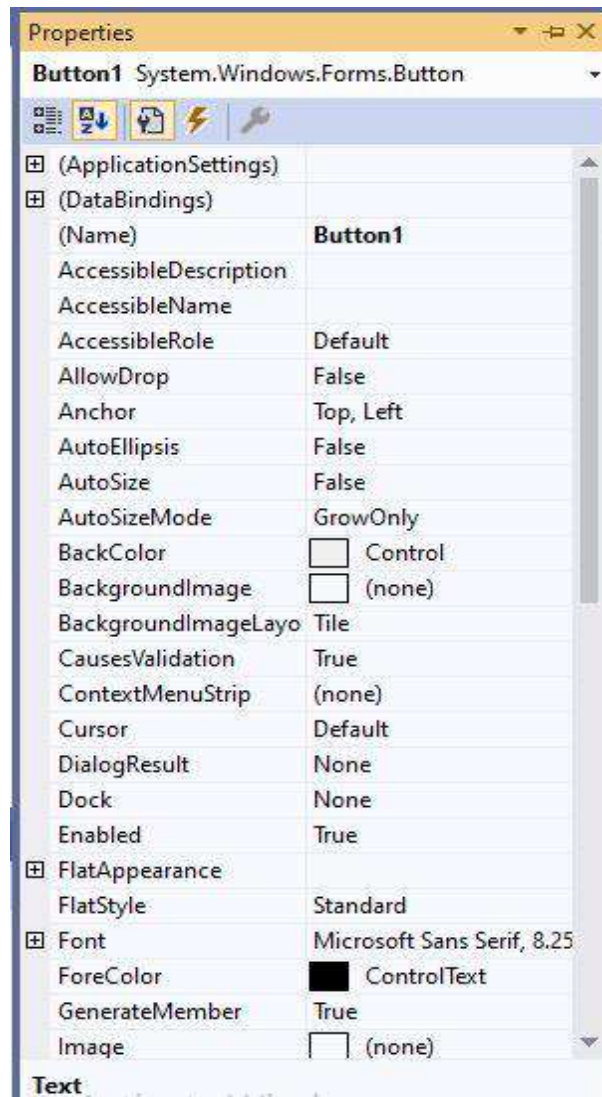
❖ **Server Explorer: -**

- The **Server Explorer** in Visual Studio is a tool window that allows you to **connect to, explore, and manage external data sources**, such as:
- Databases (SQL Server, Oracle, MySQL, etc.)
- Servers (Azure, IIS, Windows Services)
- Web services
- Event Logs



❖ **Property and Event Explorer: -**

- **Visual Studio .net Property Explorer** and **Event Explorer** are tools that help developers interact with components visually during **UI design**, particularly.
- The **Property Explorer** (called *Properties Window* in Visual Studio) allows you to **view and edit the properties** of selected UI controls **at design time**.
- Go to **View > Properties Window** or press **F4**.
- The **Event Explorer** (also part of the Properties Window) lets you **attach event handlers** (like `Click`, `Load`, etc.) to controls at design time.
- **How to Open:** Select a control (e.g., a Button) on your form.
- Open the **Properties Window (F4)**.
- Click the **lightning bolt ⚡ icon** at the top of the Properties Window. This switches to **Events View**.



❖ Class View: -

- **Class View** is a **tree-structured browser** that shows all the code elements defined in your project and referenced assemblies.
- Navigate through classes quickly
- View method/property/event definitions
- Understand class hierarchy
- Find members without opening each file
- How to Open Class view:- Go to **View > Class View**
- Or press **Ctrl + Shift + C**

MyApp

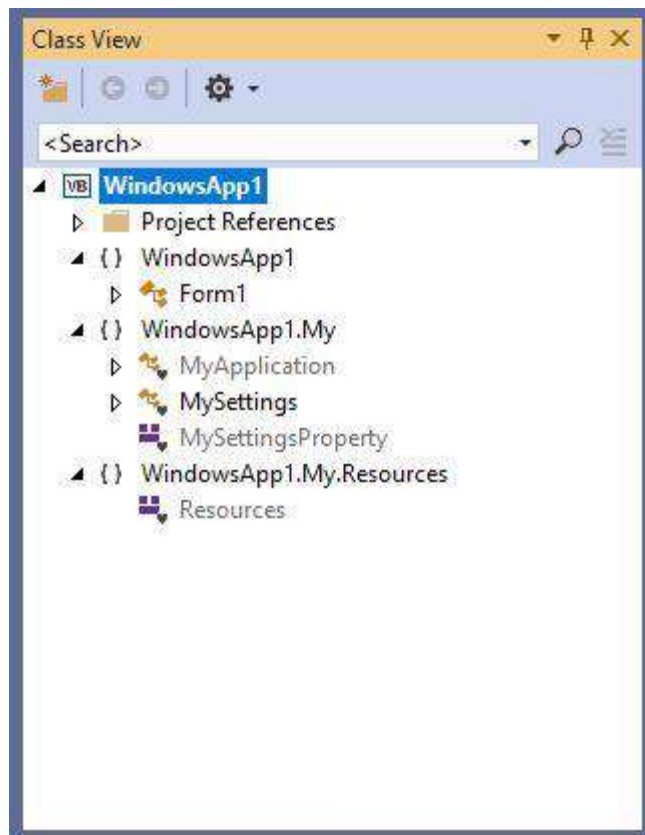
└─ **Models**

└─ **Student**

└─ **Name (Property)**

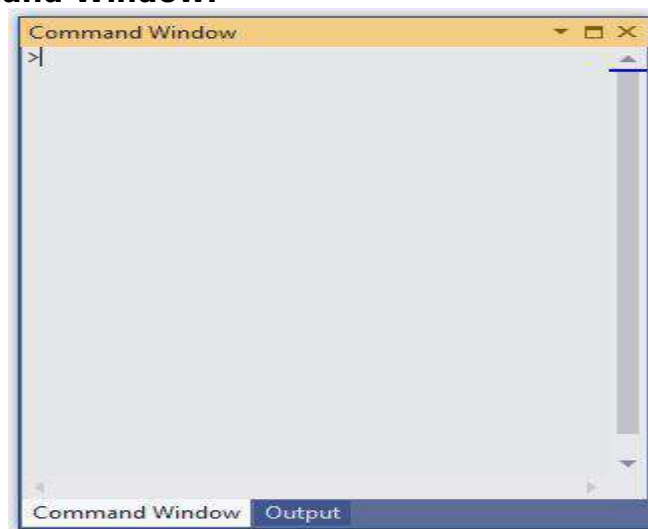
└─ **Age (Property)**

└─ **DisplayInfo() (Method)**



❖ **Command Window: -**

- **Command Window** is a powerful tool used to execute commands, run macros, and navigate code more efficiently **without using the mouse**.
- The **Command Window** is a **text-based input** console inside Visual Studio that allows you to
- Run **IDE commands** (e.g., build, open files, navigate to classes)
- Execute **debugging commands**
- Control **window layout**
- Call **macros or automation scripts**
 - **How to Open the Command Window:-** Go to **View > Other Windows > Command Window**.



○

❖ Introduction about code window:-

- The **Code Window** in Visual Studio is the **main editor** where you **write, view, and edit source code** in **C#** (or other languages). It is the core interface for coding in all types of .NET applications — such as **Windows Forms, ASP.NET, WPF, Console apps, and Class Libraries**.
- **Double-click** any .cs file (e.g., Form1.cs, Program.cs)
- Or **Right-click > View Code** from the **Solution Explorer**
- Shortcut: **F7** (when a form is selected)

❖ Understand how to write code:-

- C# (pronounced "**C-Sharp**") is a **modern, object-oriented programming language** developed by Microsoft. It's used to build desktop applications, web apps, APIs, games (with Unity), and more — usually with **Visual Studio** or **Visual Studio Code**.
 - Create Your First C# Project :- Open **Visual Studio**
 - Go to **File > New > Project**
 - Choose **Console App (.NET Core / .NET Framework)** or **Windows Forms App**
 - Name your project and click **Create**
- using System; // Namespace that includes basic classes like Console**

```
namespace MyFirstApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!"); // Output to the console
        }
    }
}
```

❖ Run Application:-

- Running a C# application depends on the **type of project** you're building — e.g., **Console App, Windows Forms, or ASP.NET**. Below is a general and specific guide using **Visual Studio**, the most popular IDE for C#.
- Run the application:- Click the **"Start" button** (green play icon).
- Press **F5** → Start application

❖ Debugging application:-

- **Debugging** is the process of running your application in a special mode to **find and fix errors** (bugs). Visual Studio provides powerful debugging tools to inspect your C# code while it runs.
- Press F5
- Or click the **green Start button** in the toolbar.
- Or go to **Debug > Start Debugging**

❖ Introduction about menus and functionality of all menu bar available tools

- **introduction to menus and menu bar functionality** in **.NET C# using Visual Studio** — especially focusing on tools and menus commonly used in C# development.
- **What is the Menu Bar in .NET C# (Visual Studio)?**
- The **menu bar** is the horizontal strip at the top of the Visual Studio IDE. It contains **menus like File, Edit, View, Project, Build, Debug, Tools, Window, and Help**, which give you access to almost every function for writing, debugging, building, testing, and managing C# projects.

❖ Menu Bar Overview & Key Functionalities

Menu Name	Description & Common Options
◆ File	Create, open, close, and save solutions/projects/files
◆ Edit	Undo, redo, cut, copy, paste, find/replace, code formatting
◆ View	Show/hide windows like Solution Explorer, Toolbox, Properties, etc.
◆ Project	Add references, manage project settings, add new forms, classes
◆ Build	Compile the solution/project, view errors
◆ Debug	Set breakpoints, start/stop debugging, view variables
◆ Tools	Access options, extensions, Get package manager
◆ Test	Run and manage unit tests
◆ Analyze	Run code analysis, performance profiling
◆ Window	Manage open windows, split views, floating/docking
◆ Help	Access documentation, check updates, about Visual Studio

Detailed Functionality of Each Menu

◆ 1. File Menu

- New → Project / File
- Open → Project / Solution
- Close Solution
- Save / Save All
- Export Template

◆ 2. Edit Menu

- Undo / Redo
- Cut / Copy / Paste
- Find and Replace
- Go To → Line, Definition, Symbol
- Advanced → Format Document, Comment/Uncomment, etc.

◆ 3. View Menu

- **Solution Explorer**
- **Server Explorer**
- **Toolbox**
- **Error List**

- **Class View**
- **Output Window**
- **Team Explorer** (for Git/TFS)

◆ **4. Project Menu**

- Add New Item / Existing Item
- Add Reference / Service / Class
- Set as Startup Project
- Project properties

◆ **5. Build Menu**

- Build Solution (Ctrl+Shift+B)
- Rebuild / Clean Solution
- Configuration Manager

◆ **6. Debug Menu**

- Start Debugging (F5)
- Step Into / Over / Out (F10, F11)
- Toggle Breakpoint (F9)
- Windows → Locals, Watch, Call Stack

◆ **7. Tools Menu**

- Options (change settings, fonts, editor themes)
- Package Manager
- Extensions and Updates
- External Tools

◆ **8. Test Menu**

- Run All Tests
- Test Explorer
- Code Coverage

◆ **9. Analyze Menu**

- Run Code Analysis
- Performance Profiler
- Code Metrics

◆ **10. Window Menu**

- New Window
- Float / Dock windows
- Reset Window Layout

◆ **11. Help Menu**

- View Help (online docs)
- Send Feedback
- About Microsoft Visual Studio

❖ Commonly Used During C# Development

Task	Menu
Create Forms/Classes	Project > Add New Item
Debugging	Debug menu
Build and Run	Build / Debug
Code Fixes	Edit > IntelliSense
Connect to Database	View > Server Explorer
Manage Packages	Tools > Package Manager
Change Theme/Settings	Tools > Options

❖ Example Workflow for C# App (Windows Forms)

1. **File > New Project** → Windows Forms App
2. **View > Toolbox** → Drag Button, TextBox
3. **Project > Add Class** → Create custom logic
4. **Build > Build Solution**
5. **Debug > Start Debugging**
6. **Tools > NuGet Package Manager** → Install external libraries
7. **Window > Reset Layout** → Restore window layout if needed

❖ Class and Event driven model

- **What is a Class?**
- A **class** in C# is a **blueprint or template** for creating objects. It defines properties (data) and methods (behavior).
- **Event-Driven Model in C# .NET**
- **Event-driven programming** is a model where the flow of the program is determined by **events** (user actions like clicks, keypresses, etc.).
- For example: Clicking a "Submit" button triggers the `Submit_Click()` method.
- Structure of an Event-Driven Program
- Control (e.g., Button)
- Event (e.g., Click)
- Event Handler (method)

❖ Basic of C#

- **Introduction to the basics of C# (C-Sharp)** — one of the most popular programming languages developed by Microsoft, primarily used with the .NET framework for building Windows, web, and mobile applications.
- **What is C#?** -- > C# is a **modern, object-oriented programming language**. Developed by Microsoft and runs on the **.NET platform**. Used for developing Windows apps, web apps, APIs, games (via Unity), and more.

❖ Advantages of OOPs: -

- OOP stands for Object-Oriented Programming.
- OOP is faster and easier to execute
- OOP provides a clear structure for the programs

- OOP systems can be easily upgraded from small to large systems.
- It is very easy to partition the work in a project based on objects.
- **Modularity and Organization:**
- OOP promotes breaking down complex systems into smaller, self-contained units called objects. This modularity makes code easier to understand, manage, and debug, as changes within one object are less likely to impact others.
- **Code Reusability:**
- Through concepts like inheritance, OOP allows for the reuse of existing code. A new class can inherit properties and behaviors from an existing class, reducing redundant code and speeding up development.
- **Encapsulation and Data Security:**
- Encapsulation bundles data and the methods that operate on that data within a single unit (an object). This hides internal implementation details and protects data from unauthorized external access, enhancing data security and integrity.
- **Flexibility and Extensibility (Polymorphism):**
- Polymorphism enables objects of different classes to be treated as objects of a common type. This provides flexibility, allowing for more generic and adaptable code that can handle various object types without extensive conditional logic. It also simplifies adding new functionalities by extending existing classes.
- **Easier Maintenance and Troubleshooting:**
- The modular and encapsulated nature of OOP makes it simpler to identify and fix issues. When a problem arises, the scope of the affected code is often limited to a specific object or class, making debugging more efficient.
- **Scalability:**
- OOP principles facilitate the development of applications that can easily grow and adapt to new requirements. The modular design and reusability allow for the seamless addition of new features and functionalities without disrupting existing code.
-

❖ Creating Class and Object

- **Class** is a blueprint for creating **objects**. A **class** keyword is used to declare a class.
- An object is an instance of a class.

```
using System;           // Declaration of class

public class Geeks      // Declaring public class
{
    public int a, b;     // field variable

    // member function or method
    public static void display()
    {
        Console.WriteLine("Class & Objects in C#");
    }

    public static void Main(String[] args)
    {
        display();
    }
}
```

- ❖ Object is a basic unit of Object-Oriented Programming and represents real-life entities. It can also refer to the instance of a class. An object of that class is

created. An object holds data and can access methods and their properties.

```
Class Program
{
    static void Main()
    {
        // Creating an object of the Person class
        Person person1 = new Person();

        // Assigning values to the object's fields
        person1.Name = "John";
        person1.Age = 30;

        // Calling a method on the object
        person1.Greet(); // Output: Hello, my name is John
    }
}
```

❖ Basic Structure of a Class in C#

```
- class Program
{
    static void Main(string[] args)
    {
        MyClass obj = new MyClass(101, "John");
        obj.DisplayInfo();
    }
}
```

❖ **Datatype:** - **Value Type:**

Data Type	Description	Example	Size
int	Integer numbers	int x = 10;	4 bytes
float	Single-precision decimal	float y = 5.5f;	4 bytes
double	Double-precision decimal	double z = 3.14;	8 bytes
char	Single character	char c = 'A';	2 bytes
bool	Boolean value (T/F)	bool isTrue = true;	1 byte
byte	Small integer (0–255)	byte b = 100;	1 byte
short	Small integer	short s = 30000;	2 bytes
long	Large integer	long l = 10000000000;	8 bytes
decimal	High-precision decimal	decimal d = 10.5m;	16 bytes

Reference Type:

Data Type	Description	Example
-----------	-------------	---------

string Sequence of characters
 object Base type of all types
 dynamic Runtime-bound type

```
string name = "C#";
object obj = 123;
dynamic d = "Hello";
```

Alias	.NET Type
int	System.Int32
float	System.Single
double	System.Double
char	System.Char
bool	System.Boolean
string	System.String

Example datatype:-
 int age = 25;
 float height = 5.9f;
 double weight = 72.5;
 decimal salary = 45000.50m;
 char grade = 'A';
 bool isPassed = true;
 string name = "John"

Datatype: -

Type	Represents
Bool	Boolean value
Byte	8-bit unsigned integer
Char	16-bit Unicode character
Decimal	28-bit precise decimal values with 28-29 significant digit
Double	64-bit double-precision floating point type
Float	32-bit single-precision floating point type
Int	32-bit signed integer type
Long	64-bit signed integer type
Sbyte	8-bit signed integer type
Short	16-bit signed integer type

UInt	32-bit unsigned integer type
Ulong	64-bit unsigned integer type
Ushort	16-bit unsigned integer type

❖ **Operators:**

- Operators are special symbols that perform operations on variables or values, known as operands. They are fundamental to performing various tasks, including mathematical calculations, comparisons, logical operations, and value assignments.

❖ **Arithmetic Operators:**

- Perform basic mathematical operations (e.g., + for addition, - for subtraction, * for multiplication, / for division, % for modulus, ++ for increment, -- for decrement).

❖ **Assignment Operators:**

- Assign values to variables (e.g., = for simple assignment, += for addition assignment, -= for subtraction assignment).

❖ **Comparison Operators (Relational Operators):**

- Compare two operands and return a Boolean result (e.g., == for equality, != for inequality, < for less than, > for greater than, <= for less than or equal to, >= for greater than or equal to).

❖ **Logical Operators:**

- Combine or modify Boolean expressions (e.g., && for logical AND, || for logical OR, ! for logical NOT).

❖ **Bitwise and Shift Operators:**

- Perform operations on the individual bits of integral types (e.g., & for bitwise AND, | for bitwise OR, ^ for bitwise XOR, ~ for bitwise complement, << for left shift, >> for right shift, >>> for unsigned right shift).

❖ **Ternary Operator (Conditional Operator):**

- A shorthand for an if-else statement, taking three operands (e.g., condition ? true_expression : false_expression).

❖ **Null-Coalescing Operator (??):**

- Provides a default value if an operand is null.

❖ **Null-Forgiving Operator (!):**

- Suppresses nullable warnings in nullable annotation contexts, indicating to the compiler that an expression is not null.

❖ **Constructor**

- **Constructor** is a special method that is **automatically called** when an object of a class is created. It is used to **initialize** objects.
- Basic Syntax of Constructor

```
public class MyClass
{
    // Field
    private string name;
```



```

// Constructor
public MyClass(string userName)
{
    name = userName;
}

// Method
public void Show()
{
    Console.WriteLine("Name: " + name);
}
}

```

Ex.

```

using System;

namespace Constructor
{
    class Program
    {
        string classname;
        int age;
        public Program()
        {
            classname = "bca";
            age = 20;
        }

        static void Main(string[] args)
        {
            Program obj = new Program();
            Console.WriteLine(obj.classname);
            Console.WriteLine(obj.age);
        }
    }
}

```

❖ Destructor

- **Destructor** in C# is used to **clean up resources** (like files, memory, database connections) when an object is no longer needed. It is automatically called by the **garbage collector** before the object is destroyed.
- C# has **automatic garbage collection**, so destructors are rarely needed.

- For Ex

```

class MyClass
{
    // Constructor
    public MyClass()
    {
        Console.WriteLine("Constructor Called");
    }

    // Destructor
}

```

```

~MyClass()
{
    Console.WriteLine("Destructor Called");
}
}

using System;

namespace ConsoleApp7
{
    public class bca
    {
        int number;
        int val;

        public void valIn(int x, int y)
        {
            number = x;
            val = y;
        }

        public void Result()
        {
            Console.WriteLine(number);
            Console.WriteLine(val);
        }

        ~bca()
        {
            Console.WriteLine("bca");
        }
    }

    public class Demo
    {
        public static void Main()
        {
            bca x = new bca();
            x.valIn(30,55);
            x.Result();
        }
    }
}

```

❖ **Abstraction:**

- **Abstraction** in C# is a fundamental **OOP concept** that hides unnecessary implementation details and shows **only the essential features** of an object. It helps reduce complexity and increase efficiency.
- **How Abstraction Works in C#**
- Abstract Classes

```

abstract class Animal
{
    public abstract void MakeSound(); // Abstract method (no body)

    public void Eat()

```

```

    {
        Console.WriteLine("This animal eats food."); // Concrete method
    }
}

```

```

class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}

```

```

class Program
{
    static void Main()
    {
        Animal a = new Dog();
        a.MakeSound(); // Output: Dog barks
        a.Eat();       // Output: This animal eats food.
    }
}

```

```

using System;

namespace ConsoleApp7
{
    abstract class Person
    {
        //public abstract void show();
        public string Firstname;
        public string Lastname;
        public int age;
        public long mobile;

        public abstract void Print();
    }

    class teacher: Person
    {
        public string qulification;
        public int salary;
        public override void Print()
        {
        }
    }

    class student: Person
    {
        public int rollno;
        public int fees;
        public override void Print()
        {
            string name = this.Firstname + " " + this.Lastname;
            Console.WriteLine(name);
            Console.WriteLine(this.age);
            Console.WriteLine(this.mobile);
            Console.WriteLine(this.rollno);
            Console.WriteLine(this.fees);
        }
    }
}

```

```

    }
}

class bca
{
    public static void Main(string[] args)
    {
        Person a = new student();
        a.Print();
    }
}
}

```

❖ Interface

- An interface defines a contract that a class or struct must to present. It outlines a set of members (methods, properties, events, indexers) that implementing types must provide.

- **For Ex.**

```

- interface IVehicle
{
    void Start(); // All methods in interface are abstract by default
}

```

```

class Car : IVehicle
{
    public void Start()
    {
        Console.WriteLine("Car has started.");
    }
}

```

Ex.

```

using System;

namespace ConsoleApp7
{
    interface Iemployee
    {
        void show();
    }

    class PartEmp : Iemployee
    {
        public void show()
        {
            //throw new NotImplementedException();

            Console.WriteLine("This is a method of interface");
        }
    }
}

class bca
{
    static void Main(string[] args)
    {
        PartEmp pte = new PartEmp();
        pte.show();
        Console.ReadLine();
    }
}

```

}

❖ Polymorphism

- **Polymorphism** is one of the four main pillars of OOP (Object-Oriented Programming) in C#. It means "**many forms**" — the **same method name** can behave **differently** based on the context.

- Types of Polymorphism in C#:
- Compile-time (Static):- Achieved using **method overloading**
- Run-time (Dynamic) :- Achieved using **method overriding**

❖ Compile-time Polymorphism (Method Overloading)

- Same method name, **different parameters** (number, type, or order)

```
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }
}
```

❖ Run-time Polymorphism (Method Overriding)

- Achieved using `virtual` and `override` keywords.

```
class Animal
{
    public virtual void MakeSound()
    {
        Console.WriteLine("Animal makes a sound");
    }
}

class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

❖ Inheritance

- **Inheritance** in C# is an object-oriented programming concept that allows a **class (child/derived class)** to inherit members (fields, methods, properties, etc.) from another **class (parent/base class)**. It promotes **code reuse** and supports **polymorphism**.

- **Syntax:-**

```
Class BaseClass
{
    // Base class members
}
```

```
class DerivedClass : BaseClass
{
    // Derived class members
}
```

Ex:- using System;

```
class Program // Base class
{
    public void Bca()
    {
        Console.WriteLine("Hello!! Bca Sem-5...");
    }
}

class Pgdca : Program // Derived class
{
    public void student()
    {
        Console.WriteLine("Pgdca sem-1...");
    }
}

class Program
{
    static void Main()
    {
        Pgdca obj = new Pgdca();

        obj.Bca(); // Inherited from Program
        obj.student(); // Defined in student
    }
}
```

❖ **Garbage Collection: -**

- The .Net Framework provides a new mechanism for releasing unreferenced objects from the memory (that is we no longer needed that object in the program), this process is called Garbage Collection (GC).
- When a program creates an Object, the Object takes up the memory. Later when the program has no more references to that Object, the Object's memory becomes unreachable, but it is not immediately freed. The Garbage Collection checks to see if there are any Objects in the heap that are no longer being used by the application. If such Objects exist, then the memory used by these Objects can be reclaimed.

❖ **Jagged Array**

- A **Jagged Array** in C# is an **array of arrays**, where each "sub-array" can have a different length. Unlike a **multidimensional array**, jagged arrays can hold rows of varying sizes.
- **Syntax** of a Jagged Array:
- `type[][] arrayName = new type[numberOfRows][];`
- **For Ex. :-**

```
using System;
```

```
class Program
```

```
{
    static void Main()
    {
        // Declare and initialize jagged array
        int[][] jaggedArray = new int[3][]; // 3 rows

        // Initialize each row with different lengths
        jaggedArray[0] = new int[] { 1, 2 };
        jaggedArray[1] = new int[] { 3, 4, 5 };
        jaggedArray[2] = new int[] { 6 };

        // Display the jagged array
        for (int i = 0; i < jaggedArray.Length; i++)
        {
            Console.WriteLine("Row " + i + ": ");
            for (int j = 0; j < jaggedArray[i].Length; j++)
            {
                Console.Write(jaggedArray[i][j] + " ");
            }
            Console.WriteLine();
        }
    }
}
```

❖ **Collection (hashtable and array list): -**

- C# provides **collection classes** in the **System.Collections** namespace to store and manage groups of related objects. Two commonly used non-generic collections are **Array list and Hashtable**.

❖ **Array List**

- A **resizable array**.
- Can store **elements of any type** (since it's non-generic, it stores `object` types).
- Allows **dynamic size**—you don't need to specify a fixed length.
- Syntax: `ArrayList list = new ArrayList();`
- **For Ex. :-**

```
using System;
```

```
using System.Collections;
```

```
class Program
```

```
{
    static void Main()

    {
        ArrayList list = new ArrayList();
```

```
list.Add(10);
list.Add("Hello");
list.Add(3.14);
list.Add(true);

Console.WriteLine("ArrayList contents:");
foreach (var item in list)
{
    Console.WriteLine(item);
}
}
```

- Can store **mixed data types**.
- Methods: `.Add()`, `.Remove()`, `.Insert()`, `.Contains()`, `.Clear()`.

❖ **Has table:-**

- Stores **key-value pairs**.
- Keys must be **unique**, but values can be duplicated.
- Uses **hashing** for quick access.
- Syntax:- `Hashtable table = new Hashtable();`
- **For Ex: -**

```
using System;
using System.Collections;
```

```
class Program
{
    static void Main()
    {
        Hashtable table = new Hashtable();

        table["id"] = 101;
        table["name"] = "John";
        table["age"] = 25;

        Console.WriteLine("Hashtable contents:");
        foreach (DictionaryEntry entry in table)
        {
            Console.WriteLine($"{entry.Key}: {entry.Value}");
        }
    }
}
```

- Fast lookups using keys.
- Methods: `.Add(key, value)`, `.Remove(key)`, `.ContainsKey()`, `.Clear()`.

❖ **Indexes (One dimension) and Property: -**

- An indexer allows an instance of a class or struct to be indexed as an array. When an indexer is defined for a class, then that class will behave like a virtual array.
- Array access operator i.e. `[ ]` is used to access the instance of the class which uses an indexer.
- A user can retrieve or set the indexed value without pointing to an instance or a type member.
- The main difference between Indexers and Properties is that the accessors of the

indexers (get and set) take parameters, whereas properties do not.

- **Property: -**
- Properties are the special types of class members that provide a flexible mechanism to read, write, or compute the value of a private field. Properties function like public data members but are accessors which makes easy data access.
- Properties also support encapsulation and abstraction through **"get"** and **"set"** methods for accessing and modifying them. There are two main reasons to use properties.
- The block of "set" and "get" is known as Accessors. It is very essential to restrict the accessibility of the property. There are two types of accessors **i.e. get accessors and set accessors.**
- **Syntax Of Indexes: -**

```
[Access_modifier] [Return_type] this [Parameter_list]
{
    get
    { // get block code }
    set
    { // set block code }
}
```

- **Syntax Of Property: -**

```
<access_modifier><return_type><property_name>
{
    get{//body}
    set{//body}
}
```

- **Ex. Of Property: -**

// Demonstrating issues with public and private fields

```
using System;

public class C1
{
    // Public data members (No restrictions)
    public int rn;
    public string name;
}

public class BCA
{
    public static void Main(string[] args)
    {
        // Creating an object of class C1
        C1 obj = new C1();

        // Setting values to public data members
        obj.rn = 1001;
        obj.name = "abc";

        // Directly modifying private field is not possible
    }
}
```

```

        //obj.mark = 0; // Error

        Console.WriteLine("Name: {0} \nRoll No: {1}"
            , obj.name, obj.rn);
    }
}

```

- **Ex. Of Indexes: -**

```

// Use of Indexers in C#
using System;

class BCA
{
    private string[] values = new string[3];

    // Indexer declaration
    public string this[int index]
    {
        // Getter
        get
        {
            return values[index];
        }
        // Setter
        set
        {
            values[index] = value;
        }
    }
}

class Program
{
    static void Main()
    {
        BCA o = new BCA();

        // Assign values using the indexer
        o[0] = "C";
        o[1] = "C++";
        o[2] = "C#";

        // Access values using the indexer
        Console.WriteLine("First value: " + o[0]);
        Console.WriteLine("Second value: " + o[1]);
        Console.WriteLine("Third value: " + o[2]);
    }
}

```

❖ Delegates and Event Multicasting

- C# delegates are similar to pointers to functions, in C or C++.
- A delegate is a reference type variable that holds the reference to a method.
- The reference can be changed at runtime.
- Delegates are especially used for implementing events and the call-back methods.
- All delegates are implicitly derived from the System.Delegate class.

Declaring Delegates

- Delegate declaration determines the methods that can be referenced by the delegate.
- A delegate can refer to a method, which have the same signature as that of the delegate.

For example, consider a delegate:

```
public delegate int MyDelegate (string s);  
For Ex:-  
using System;  
public class Program  
{  
    public delegate void MyDelegate(string message);  
    // Step 1: Declare delegate  
    public static void Display(string msg)  
    {  
        Console.WriteLine("Display: " + msg);  
    }  
    public static void Show(string msg)  
    {  
        Console.WriteLine("Show: " + msg);  
    }  
    public static void Main()  
    {  
        MyDelegate del = Display; // Step 2: Assign method  
        del += Show; // Step 3: Multicast (add another method)  
        del("Hello from delegate!"); // Step 4: Call all methods in the invocation list  
    }  
}
```

- Event **multicasting** allows **multiple methods** to subscribe to a single event and be notified when the event is raised.

- For Ex:-

```
using System;
public class Publisher
{
    public delegate void Notify();    // Declare a delegate
    public event Notify OnPublish;    // Declare an event

    public void Publish()
    {
        Console.WriteLine("Publishing message...");
        OnPublish?.Invoke();        // Raise the event
    }
}

public class Subscriber
{
    public void Message1()
    {
        Console.WriteLine("Message 1 received.");
    }

    public void Message2()
    {
        Console.WriteLine("Message 2 received.");
    }
}

class Program
{
    static void Main(String[] args)
    {
        Publisher pub = new Publisher();
        Subscriber sub = new Subscriber();

        // Subscribing multiple methods (multicasting)
        pub.OnPublish += sub.Message1;
        pub.OnPublish += sub.Message2;

        pub.Publish();    // Trigger the event
    }
}
```

❖ Exception handling

- Exception handling in C# is a mechanism to handle **runtime errors** in a graceful manner using the `TRY`, `catch`, `finally`, and `throw` keywords.
- Syntax: -

```
try
{
    // Code that might cause an exception
}
catch (ExceptionType ex)
{
    // Code to handle the exception
}
```

```

finally
{
    // Code that will always execute
}

```

- **try** – Wraps code that may cause an exception.
- **catch** – Catches and handles the exception. Handles specific types of exceptions if they occur within the try block. Multiple catch blocks can be used to handle different exception types.
- **finally** – (Optional) Executes regardless of whether an exception occurred. Contains code that is always executed, regardless of whether an exception occurred or was caught. This is typically used for cleanup operations (e.g., closing file streams).
- **throw** – Used to manually throw an exception or rethrow a caught exception. Used to explicitly raise an exception. This can be a new instance of an existing exception type or a custom exception.
- **For Example:**

```

using System;

class Program
{
    static void Main(String[] args)
    {
        int a = 10, b = 0;

        try
        {
            int result = a / b;
            Console.WriteLine("Result: " + result);
        }
        catch (DivideByZeroException ex)
        {
            Console.WriteLine("Error: " + ex.Message);
        }
        finally
        {
            Console.WriteLine("Execution completed.");
        }
    }
}

```

❖ String handling function

- **String handling functions** are provided by the `String` class in the `System` namespace. These functions allow you to perform operations such as comparing, modifying, searching, and formatting strings.
- Commonly Used String Handling Functions in C#

Function	Description
<code>Length</code>	Returns the number of characters in the string.
<code>ToUpper()</code>	Converts string to uppercase.
<code>ToLower()</code>	Converts string to lowercase.
<code>Substring(start, length)</code>	Returns a substring starting at a specific index.
<code>IndexOf("text")</code>	Returns the index of the first occurrence of a substring.

Function	Description
LastIndexOf("text")	Returns the index of the last occurrence of a substring.
Contains("text")	Returns true if the substring exists in the string.
Replace("old", "new")	Replaces all occurrences of a substring with another.
Trim()	Removes leading and trailing whitespaces.
Split('char')	Splits the string into an array based on a character.
Join(separator, array)	Joins an array of strings into a single string with a separator.
Equals()	Compares two strings for equality.
Compare(str1, str2)	Compares two strings and returns an integer (-1, 0, 1).
StartsWith("text")	Checks if the string starts with a given substring.
EndsWith("text")	Checks if the string ends with a given substring.
Insert(index, "text")	Inserts a string at a specific index.
Remove(start, count)	Removes a specified number of characters from a string.

- For Example: -

```
using System;

class Program
{
    static void Main(String[] args)
    {
        string str = "Hello, World! ";
        Console.WriteLine("Original: " + str);
        Console.WriteLine("Trimmed: " + str.Trim());
        Console.WriteLine("Upper: " + str.ToUpper());
        Console.WriteLine("Lower: " + str.ToLower());
        Console.WriteLine("Substring: " + str.Substring(2, 5));
        Console.WriteLine("Contains 'World': " + str.Contains("World"));
        Console.WriteLine("Replace: " + str.Replace("World", "C#"));
        Console.WriteLine("Length: " + str.Length);
        Console.WriteLine("Starts with 'He': " + str.StartsWith("He"));
    }
}
```

❖ Creating function with all types

- In C#, you can create functions (also called **methods**) that use **various types**—such as built-in data types, reference types, collections, and custom types. Below is a complete example that shows **functions using all major C# data types**.

- **Syntax:**

```
<access-specifier><return-type>Function Name(<parameters>)
{
    // Function body
    // return statement
}
```

```
Ex. Public int show(int a, int b)
{
    Int c=a+b;    return c;
}
```

- There are three types of function:
 - 1) Using no parameter and return type
 - 2) Using parameter but no return type
 - 3) Using parameter and return type

- For Ex.

```
using System;
using System.Collections.Generic;

class Program
{
    // Function with int
    static int Add(int a, int b)
    {
        return a + b;
    }
    // Function with float
    static float Multiply(float x, float y)
    {
        return x * y;
    }
    // Function with double
    static double Divide(double a, double b)
    {
        return a / b;
    }
    // Function with string
    static string Greet(string name)
    {
        return "Hello, " + name;
    }
    // Function with char
    static void PrintChar(char ch)
    {
        Console.WriteLine("Character is: " + ch);
    }
    // Function with bool
    static bool IsEven(int number)
    {
        return number % 2 == 0;
    }
    // Function with array
    static int SumArray(int[] numbers)
    {
        int sum = 0;
```

```

        foreach (int num in numbers)
            sum += num;
        return sum;
    }
    // Function with List
    static void DisplayList(List<string> items)
    {
        foreach (var item in items)
            Console.WriteLine(item);
    }
    // Function with object
    static void PrintObject(object obj)
    {
        Console.WriteLine("Object value: " + obj.ToString());
    }
    // Function with custom class
    class Student
    {
        public string Name;
        public int Age;
    }
    static void DisplayStudent(Student s)
    {
        Console.WriteLine("Student: " + s.Name + ", Age: " +
s.Age);
    }
    // Main function
    static void Main(String[] args)
    {
        Console.WriteLine("Add: " + Add(10, 5));
        Console.WriteLine("Multiply: " + Multiply(3.5f, 2f));
        Console.WriteLine("Divide: " + Divide(10.0, 2.0));
        Console.WriteLine(Greet("BCA Sem-5!!!"));
        PrintChar('A');
        Console.WriteLine("Is Even: " + IsEven(4));
        Console.WriteLine("Array Sum: " + SumArray(new int[]
{ 1, 2, 3 }));
        List<string> fruits = new List<string> { "Apple",
"Banana", "Mango" };
        DisplayList(fruits);
        PrintObject(123); // can be any object
        Student stu = new Student() { Name = "BCA College",
Age = 25 };
        DisplayStudent(stu);
    }
}

```


} }